TECHNICAL REPORT

AI-Powered Insurance Recommendation System

LangGraph • GPT-5.4 • Deterministic Scoring • Evaluation Harness

Document Overview

This report documents the architecture, design decisions, evaluation strategy, and limitations of a stateful graph-based insurance recommendation system built with LangGraph. The system routes user requests through a planner, retrieval, scoring, and composition pipeline — combining deterministic business logic with LLM-powered planning for reliable, auditable recommendations.

Confidential

1. System Architecture

The system is implemented as a stateful graph-based workflow orchestrated with LangGraph. A user request enters through main.py, which initializes a shared AgentState object and invokes the compiled graph. The operational flow moves through five primary nodes, with conditional routing that enables efficient handling of both standard and edge-case requests.

1.1 Workflow Architecture Diagram

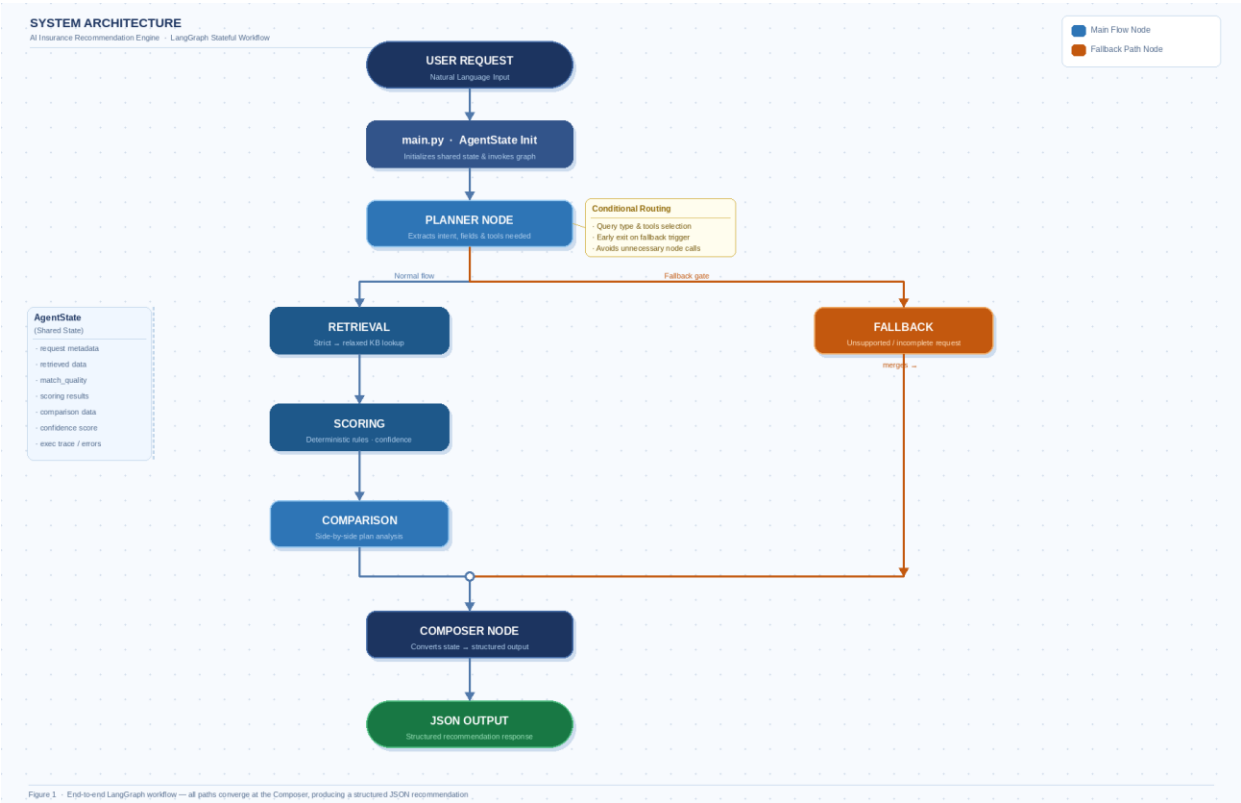

Figure 1 — End-to-end system flow from user request to JSON output

1.2 Node Descriptions

The Planner is responsible for request understanding and task selection. It extracts structured fields such as industry, region, budget, priority, comparison targets, and query type. It selects the required tools and writes those decisions into shared state. A conditional check avoids unnecessary downstream tool calls when clarification or fallback is already required.

The Retrieval node resolves request attributes against the embedded knowledge base. It performs a strict lookup first, then retries with relaxed matching when no customer profile is found. It writes back match_quality and retrieval_reason, making partial matches observable and easier to evaluate.

The Scoring node applies deterministic business rules to candidate packages. It calculates a score for each plan based on coverage, budget, industry, region, priority, dependents ratio, and price pressure. It handles conflict cases such as low budget versus best coverage, and lowers confidence when the top score gap is narrow.

The Comparison node performs side-by-side comparison when the planner detects comparison intent. It reuses scoring logic, then produces a recommendation reason and a business-oriented comparison summary.

The Composer serves as the terminal node for all paths — both successful and fallback. It converts shared state into the final structured JSON output without altering the output schema.

2. Framework Justification

LangGraph is a strong fit for this system because the workflow is stateful, branching, and partially deterministic. It provides a native graph abstraction, supports conditional edges cleanly, and makes node-level testing straightforward.

2.1 Why Not Alternative Frameworks?

CrewAI is more role-oriented and optimized for teams of conversational agents. That pattern is useful when multiple persona-based agents debate or collaborate, but it is not ideal for a workflow where deterministic tools must remain in control of scoring and fallback rules.

AutoGen is primarily conversation-centric. The coordination overhead is higher than needed for a compact pipeline with a single shared state and explicit routing — adding complexity without clear benefit.

A custom orchestration layer would work, but it would recreate state management, branching, checkpointing, and graph visualization patterns that LangGraph already provides, slowing development and increasing maintenance cost.

2.2 Trade-offs

LangGraph introduces a learning curve, and graph-based systems can feel more complex than a linear script for very small projects. The added structure is justified here because the workflow already includes routing, retry behavior, fallback paths, and evaluation requirements.

3. Model Selection Justification

The system is configured to use gpt-5.4 as the only LLM model for both planning and final composition. This keeps deployment simple, reduces configuration drift, and guarantees that both model-driven steps operate at the same capability level.

3.1 Deterministic Scoring Layer

The scoring layer is intentionally deterministic Python rather than an LLM. Insurance recommendation logic needs to be predictable, auditable, and easy to test. Deterministic scoring eliminates hallucination risk, makes trade-offs explicit, and removes token cost from repeated ranking operations.

3.2 Hybrid Approach Comparison

Approach	Strengths	Weaknesses
LLM Everywhere	Flexible, natural-language rich, quick to prototype	Higher cost, more latency, harder to audit, hallucination risk
Hybrid Approach	Predictable scoring, lower cost, easier testing, clearer control flow	More engineering effort, split logic across model and code

Confidential

Using an LLM in every step would increase cost, latency, and unpredictability, and would make debugging harder because reasoning would be implicit inside model outputs instead of encoded in transparent business rules. The hybrid design is therefore a deliberate systems decision rather than a compromise.

4. Evaluation Strategy

Evaluation is implemented with a custom evaluation harness rather than an external observability platform. The project required local, reproducible, scenario-based correctness checks more than hosted tracing infrastructure. The harness runs the real workflow end-to-end, validates output against benchmark expectations, and saves a structured artifact to outputs/eval_report.json.

4.1 Scenario Coverage

Scenario	Type	Coverage
V1–V5	Visible scenarios	Balanced rec, cost-focus, comparison, missing-info, explanation
H1–H8	Hidden-style scenarios	Conflicting constraints, invalid targets, tool failure, overconfidence

4.2 Metrics Evaluated

- Overall score and pass rate across all scenarios
- Separate visible score and hidden score
- Plan correctness when an expected plan set exists
- Reasoning presence and expected tool usage
- Clarification behavior for underspecified requests
- Fallback correctness for unsupported or invalid requests
- Execution trace and confidence presence
- Confidence guard checks for risky scenarios
- Duplicate or unnecessary tool-use detection
- Retry trace detection and hallucination detection on plan_name

4.3 Edge Case Handling

Edge cases for tool failure, model overconfidence, and repeated tool calls are covered as auxiliary unit checks in _run_auxiliary_checks() rather than full end-to-end scenarios, since they require direct tool injection to trigger reliably. This design separates integration-level scenario testing from unit-level tool behavior validation.

Confidence scoring is validated both indirectly and directly. The scoring engine reduces confidence when data is missing, when business constraints conflict, and when the gap between the top two plans is narrow. Hallucination detection verifies that the recommended plan_name, when present, is one of Basic, Standard, or Premium.

Confidential

5. Limitations & Recommended Next Steps

5.1 Current Limitations

The current system does not support real-time pricing. All pricing is embedded in a fixed local dataset, so recommendations are illustrative rather than production-grade. The dataset is intentionally small — this keeps the project deterministic and easy to evaluate, but limits coverage across industries, geographies, and plan variants.

The evaluation harness is strong for local correctness testing but is not a replacement for production observability. The system currently lacks dedicated statistical calibration for confidence scores beyond the deterministic heuristics already in place.

The planner's optimality is not yet scored as a standalone metric, and multilingual support for Arabic requests is not yet implemented despite the system's target market.

5.2 Recommended Next Steps

- Add vector search or retrieval augmentation for larger policy and customer corpora
- Add multilingual support so Arabic and English requests can be handled consistently
- Introduce external pricing and policy connectors with appropriate guardrails
- Add calibration logic for confidence scoring beyond deterministic heuristics
- Expand the evaluation suite with adversarial and regression-focused scenarios
- Integrate production observability via Langfuse or OpenTelemetry for tracing and monitoring

5.3 Architecture Summary

Component	Technology	Rationale
Orchestration	LangGraph	Stateful, conditional routing, graph abstraction
Planning & Composition	GPT-5.4	Natural language understanding, structured output
Scoring	Deterministic Python	Auditable, predictable, hallucination-free
Evaluation	Custom Harness	Local reproducibility, scenario-based validation
State Management	AgentState (shared)	Explicit, inspectable, no hidden side effects

End of Technical Report

Confidential